```
Ext.define('Hammerhead.grid.receiving.AssetItemGrid', {
    extend: 'Hammerhead.grid.GridField',
    alias: 'widget.assetitemgrid',
    requires: ['Hammerhead.store.receiving.OrderedItems'],
    allowTypeSelection: true,
    allowIndicators: false,
    defaultType: 1,
    reqMark: '<span class="requiredMark">*</span>',
    viewConfig: {
        deferEmptyText: false,
        emptyText: '<div class="x-grid-empty">No Asset Items to Display</div>'
    },
    initComponent: function() {
        var me = this, conditions = Hammerhead.util.getStore('Hammerhead.store.receivi
ng.Conditions');
        me.columns = [{
            header: 'r_lid',
            dataIndex:'r_lid',
            hidden: true
        },{
                xtype: 'rownumberer'
        },{
                header: me.reqMark + 'Serial Number',
            dataIndex: 'serialnum',
            errorSummary:false,
            msgTarget: 'none',
            editor: {
                xtype: 'textfield',
                allowBlank: false,
                validator: me.refNumValidator,
                maxLength: 64,
                enforceMaxLength: true
            }
        },{
                header: 'Condition Code',
                dataIndex: 'conditioncode',
                editor: {
                        xtype: 'combo',
                        displayField: 'description',
                        valueField: 'value',
                        store: conditions,
                        listConfig: {minWidth: 200},
                        queryMode: 'local'
                },
                renderer: function(v) {
                        var id = Ext.isObject(v) ? v.value : v;
                        return id != null && id != " " ? conditions.findRecord('value'
, id).get('description') : '';
                }
        },{
            header: 'New Part #',
            dataIndex: 'partnum',

            disabled: me.disc !== Hammerhead.constants.RECEIVING_DISCREPANCY.SUBSTITUT
E,
            editor: {
                xtype: 'textfield',
                maxLength: 50,
                disabled: me.disc !== Hammerhead.constants.RECEIVING_DISCREPANCY.SUBST
ITUTE,
                enforceMaxLength: true
            }
        },{
            header: 'Asset Barcode',
            dataIndex: 'assetbarcode',
```

```
            editor: {
                xtype: 'textfield',
                maxLength: 50,
                enforceMaxLength: true
            }
        },{
            header: 'Asset Comment',
            dataIndex: 'asset_comment',
            editor: {
                xtype: 'textfield',
                maxLength: 50,
                enforceMaxLength: true
            }

        }];

        me.extraButtons = ['-',{
            text: 'Add Bulk',
            handler: me.onAddBulk,
          //  hidden: me.hideEditButtons || me.hideBulk,
            scope: me
        }];
        me.callParent(arguments);
        me.getDockedComponent(1).getComponent(0).setDisabled(me.received);
        me.getDockedComponent(1).getComponent(4).setDisabled(me.received);
    },
    onAddBulk: function() {
        var me = this;
           Ext.create('Hammerhead.window.BulkUploadWindow', {
               title: 'Bulk Asset Upload',
               url: Hammerhead.data.BASE_URL + 'receiving/bulkassetupload',
               params: {receivingLineId: me.line_id},
               templateUrl: Hammerhead.data.APP_URL + "/documents/serial.csv",
               instructionUrl:Hammerhead.constants.EXTERNAL_URLS.BULK_ASSET_HELP,
               showHelpText: true,
               listeners: {
                'uploadComplete': me.handlePostUpload,
                scope: me
               }
           }).show();
    },
    handlePostUpload: function() {
        var me = this;

        // Immediately fire back to read the file just posted to the server.
        Ext.Ajax.request({
            url: Hammerhead.data.BASE_URL + 'receiving/' + 'assetsfromupload.json',
            callback: me.processBulkAssets,
            method: 'GET',
            scope: me
        });
    },
    processBulkAssets: function(opts, success, resp) {
        var me = this;
        var store = me.getStore();
        store.removeAll();
                var rawRecs = resp.responseObject.assets;
                var clientRecs = [];
                var r_lid = me.line_id;
                Ext.Array.each(rawRecs, function(rawRec) {
                        var realRec = {}
                        realRec.serialnum = rawRec.serialNum;
                        realRec.asset_comment = rawRec.assetComment;
                        realRec.conditioncode = !Ext.isEmpty(rawRec.conditionCode) ? r
awRec.conditionCode : ' ';
```

```
                        realRec.assetbarcode = rawRec.assetBarCode;
                        realRec.r_lid=r_lid;
                        clientRecs.push(realRec);
                });
                store.loadRawData(clientRecs);
    },
    onAddButton: function() {
        var me = this,rowEditor = me.getRowEditor(), newIdx = me.store.getCount();
        if (newIdx !== 0){
        var serialnumValue = me.store.data.items[newIdx-1].data.serialnum;
        if (serialnumValue === null || Ext.isEmpty(serialnumValue)){
                Ext.Msg.alert('Error', 'Serial Number on line number: ' +newIdx+ ' can
not be blank.<BR/>Please press update or cancel to finish editing the row');
                return;
                }
        }

        rowEditor.cancelEdit();
        me.store.add(Ext.create('Hammerhead.model.receiving.AssetItem', {
                id: newIdx,
                r_lid: me.line_id,
                serialnum: null,
                conditioncode: null,
                partnum: null,
                asset_comment: null
        }));
        rowEditor.startEdit(newIdx,1);
    },
        onEdit: function(roweditiing,e) {
        var me = this, lastRow = me.store.count()-1;
        me.callParent(arguments);
        if (e.rowIdx === lastRow) {
            me.onAddButton();
        }
    },
    onRemoveButton: function() {
        var me = this;
        me.callParent();
        me.getView().refresh();
    },
    onBeforeEdit: function(roweditiing, e) {
        var me = this;

        return (!me.received);
    },
    onSelectionChange: function(sm, recs) {
        var me = this;
        if (me.allowAddRemove){
            this.getDockedComponent(1).getComponent(2).setDisabled(recs.length < 1 ||
me.received);
        }
    },
    onCancelEdit: function(plugin, e) {
        if(e.record.phantom || Ext.isEmpty(e.record.data.serialnum)) {
            this.store.remove(e.record);
        }
    }
});
```